

ASSIGNMENT 01 – SOFTWARE ARCHITECTURE AND DESIGN

Name: Nguyễn Quang Hùng – Studentcode: B22DCCN365

Course: Software Architecture and Design

Topic: Book Store Web System

Technology: Django, MySQL, Visual Paradigm

1. Introduction

This project aims to design and implement a Book Store Web System using three architectural styles: Monolithic Architecture, Clean Architecture, and Microservices Architecture. The system is developed using Django framework with MySQL as the database. UML diagrams are designed using Visual Paradigm to clearly illustrate system structure and architecture.

2. System Requirements

2.1 Entities

- **Customer:** id, name, email, password
- **Book:** id, title, author, price, stock
- **Cart:** id, customer_id, created_at
- **CartItem:** id, cart_id, book_id, quantity

2.2 Functional Requirements

- Customer registration and login
- View book catalog
- Add books to shopping cart
- View shopping cart contents

3. UML Design (Visual Paradigm)

3.1 Class Diagram

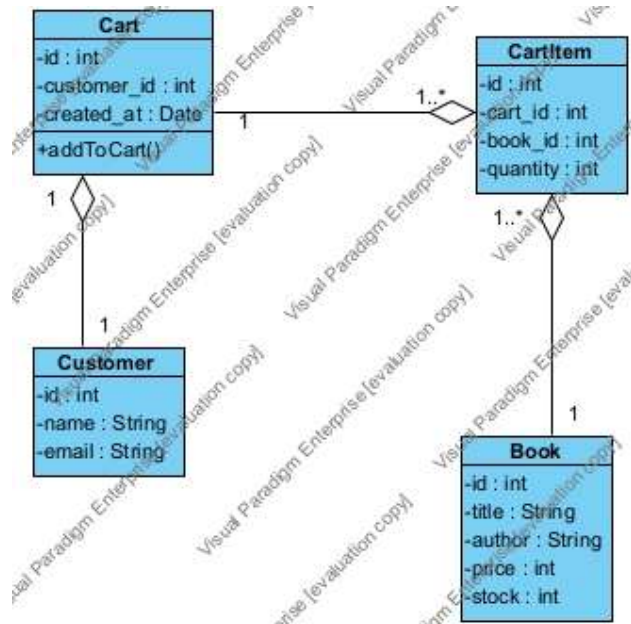
Classes and attributes:

- Customer(id, name, email, password)
- Book(id, title, author, price, stock)
- Cart(id, created_at)
- CartItem(id, quantity)

Relationships:

- Customer 1 – 1 Cart

- Cart 1 – * CartItem
- Book 1 – * CartItem



3.2 MVC Layer Diagram (Monolithic)

- **Model:** Customer, Book, Cart, CartItem
- **View:** register.html, login.html, book_list.html, cart.html
- **Controller:** Django Views (RegisterView, LoginView, BookListView, AddToCartView, CartDetailView)

3.3 Clean Architecture Diagram

Layers (from inner to outer):

- Domain (Entities)
- Use Cases (Business logic)
- Interfaces (Repository interfaces)
- Infrastructure (Database implementation)
- Framework (Django views, URLs)

3.4 Microservices Architecture Diagram

- customer-service
- book-service
- cart-service

Services communicate via RESTful APIs using JSON over HTTP.

4. Implementation

4.1 Version A – Monolithic Django

Project Structure

monolith/

|— bookstore/

|— accounts/

|— books/

|— cart/

|— manage.py

Example Model (Book)

```
class Book(models.Model):  
    title = models.CharField(max_length=255)  
    author = models.CharField(max_length=255)  
    price = models.FloatField()  
    stock = models.IntegerField()
```

4.2 Version B – Clean Architecture Django

Project Structure

clean/

|— domain/

|— usecases/

|— interfaces/

|— infrastructure/

|— framework/

Domain Entity Example

```
class Book:  
    def __init__(self, id, title, author, price, stock):  
        self.id = id
```

```
self.title = title  
self.author = author  
self.price = price  
self.stock = stock
```

4.3 Version C – Microservices Django

Services

- customer-service: manage customer data
- book-service: manage book catalog
- cart-service: manage shopping cart

API Example

- GET /api/books
- POST /api/cart/add

5. Database Design (MySQL)

```
CREATE TABLE customer (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  name VARCHAR(100),  
  email VARCHAR(100),  
  password VARCHAR(255)  
);
```

```
CREATE TABLE book (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  title VARCHAR(255),  
  author VARCHAR(255),  
  price FLOAT,  
  stock INT  
);
```

```
CREATE TABLE cart (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    customer_id INT,  
    created_at DATETIME,  
    FOREIGN KEY (customer_id) REFERENCES customer(id)  
);
```

```
CREATE TABLE cart_item (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    cart_id INT,  
    book_id INT,  
    quantity INT,  
    FOREIGN KEY (cart_id) REFERENCES cart(id),  
    FOREIGN KEY (book_id) REFERENCES book(id)  
);
```

6. Conclusion

This assignment demonstrates the application of different software architectures in a real-world web system. Through UML design and Django implementation, the system highlights architectural differences and design principles in Monolithic, Clean Architecture, and Microservices approaches.

7. Submission Notes

- UML diagrams must be drawn using Visual Paradigm
- Source code organized into /monolith, /clean, /micro folders
- Report exported as PDF

8. Full Implementation Guide (Step-by-Step)

This section provides **Project Structure and Example Code** for all three versions.

8.1 VERSION A – MONOLITHIC DJANGO (FULL CODE)

Project structure

monolith/

```
|— bookstore/
|   |— settings.py
|   |— urls.py
|— accounts/
|   |— models.py
|   |— views.py
|   |— urls.py
|— books/
|   |— models.py
|   |— views.py
|   |— urls.py
|— cart/
|   |— models.py
|   |— views.py
|   |— urls.py
|— manage.py
```

accounts/models.py

```
from django.db import models
```

```
class Customer(models.Model):
    name = models.CharField(max_length=100)
    email = models.EmailField(unique=True)
    password = models.CharField(max_length=255)
```

books/models.py

```
from django.db import models
```

```
class Book(models.Model):
```

```
title = models.CharField(max_length=255)
author = models.CharField(max_length=255)
price = models.FloatField()
stock = models.IntegerField()
```

cart/models.py

```
from django.db import models
from accounts.models import Customer
from books.models import Book
```

```
class Cart(models.Model):
    customer = models.OneToOneField(Customer, on_delete=models.CASCADE)
    created_at = models.DateTimeField(auto_now_add=True)
```

```
class CartItem(models.Model):
    cart = models.ForeignKey(Cart, on_delete=models.CASCADE)
    book = models.ForeignKey(Book, on_delete=models.CASCADE)
    quantity = models.IntegerField()
```

books/views.py

```
from django.shortcuts import render
from .models import Book
```

```
def book_list(request):
    books = Book.objects.all()
    return render(request, 'book_list.html', {'books': books})
```

cart/views.py

```
from django.shortcuts import redirect, render
from .models import Cart, CartItem
from books.models import Book
```

```

def cart_detail(request):
    cart = Cart.objects.get(customer_id=1)
    items = CartItem.objects.filter(cart=cart)
    return render(request, 'cart.html', {'items': items})

def add_to_cart(request, book_id):
    cart, _ = Cart.objects.get_or_create(customer_id=1)
    book = Book.objects.get(id=book_id)
    CartItem.objects.create(cart=cart, book=book, quantity=1)
    return redirect('cart_detail')

```

8.2 VERSION B – CLEAN ARCHITECTURE (FULL CODE)

Structure

clean/

```

├── domain/entities.py
├── usecases/add_to_cart.py
├── interfaces/repositories.py
├── infrastructure/mysql_repository.py
└── framework/views.py

```

domain/entities.py

```

class Book:
    def __init__(self, id, title, author, price, stock):
        self.id = id
        self.title = title
        self.author = author
        self.price = price
        self.stock = stock

```

```

class Cart:

```

```
def __init__(self, id):
```

```
    self.id = id
```

```
    self.items = []
```

```
def add_item(self, book, quantity):
```

```
    self.items.append((book, quantity))
```

usecases/add_to_cart.py

```
class AddToCartUseCase:
```

```
    def __init__(self, cart_repo):
```

```
        self.cart_repo = cart_repo
```

```
def execute(self, cart_id, book, quantity):
```

```
    cart = self.cart_repo.get(cart_id)
```

```
    cart.add_item(book, quantity)
```

```
    self.cart_repo.save(cart)
```

interfaces/repositories.py

```
class CartRepository:
```

```
    def get(self, cart_id):
```

```
        raise NotImplementedError
```

```
    def save(self, cart):
```

```
        raise NotImplementedError
```

infrastructure/mysql_repository.py

```
from interfaces.repositories import CartRepository
```

```
from domain.entities import Cart
```

```
class MySQLCartRepository(CartRepository):
```

```
    def get(self, cart_id):
```

```
        return Cart(cart_id)
```

```
def save(self, cart):  
    pass
```

8.3 VERSION C – MICROSERVICES (FULL CODE)

cart-service/views.py

```
from rest_framework.decorators import api_view  
from rest_framework.response import Response
```

```
@api_view(['POST'])  
def add_to_cart(request):  
    return Response({'message': 'Book added to cart'})
```

book-service/views.py

```
from rest_framework.decorators import api_view  
from rest_framework.response import Response
```

```
@api_view(['GET'])  
def list_books(request):  
    return Response([  
        {'id': 1, 'title': 'Clean Architecture', 'price': 20.5}  
    ])
```

customer-service/views.py

```
from rest_framework.decorators import api_view  
from rest_framework.response import Response
```

```
@api_view(['POST'])  
def register(request):  
    return Response({'status': 'registered'})
```

9. MySQL DATABASE SCRIPT (ALL VERSIONS)

```
CREATE TABLE customer (
```

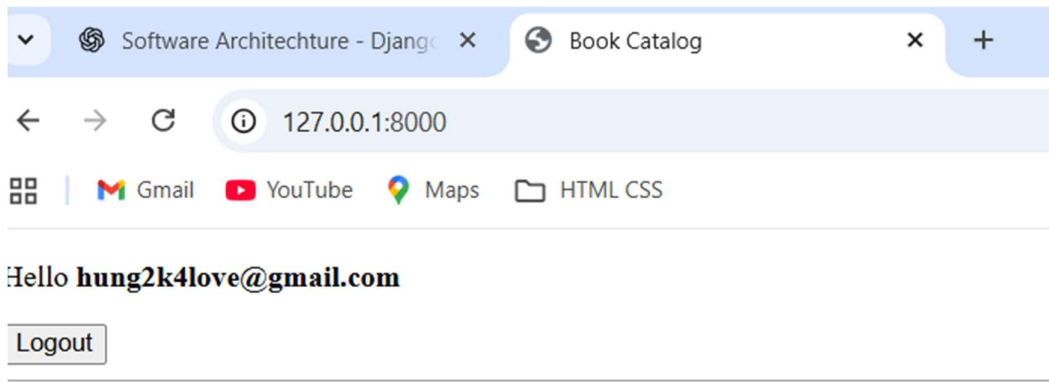
```
id INT AUTO_INCREMENT PRIMARY KEY,  
name VARCHAR(100),  
email VARCHAR(100),  
password VARCHAR(255)  
);
```

```
CREATE TABLE book (  
id INT AUTO_INCREMENT PRIMARY KEY,  
title VARCHAR(255),  
author VARCHAR(255),  
price FLOAT,  
stock INT  
);
```

```
CREATE TABLE cart (  
id INT AUTO_INCREMENT PRIMARY KEY,  
customer_id INT,  
created_at DATETIME  
);
```

```
CREATE TABLE cart_item (  
id INT AUTO_INCREMENT PRIMARY KEY,  
cart_id INT,  
book_id INT,  
quantity INT  
);
```

- RESULT:



Book Catalog

Ông già và biển cả - 150000.0\$ [Add to Cart](#)

[View Cart](#)

10. FINAL SUBMISSION CHECKLIST

- UML diagrams (Visual Paradigm)
- Full Django code (3 versions)
- MySQL SQL scripts
- PDF report

LINK GITHUB: <https://github.com/diggoryQH/proj1>